



Casa abierta al tiempo

UNIVERSIDAD AUTÓNOMA METROPOLITANA
Unidad Cuajimalpa

División de Ciencias Naturales e Ingeniería
Licenciatura en Ingeniería en Computación

Punto de Venta Vinculado con un
Bot de Discord

Proyecto Terminal

Autor:

Álvaro Arturo Ramírez Arriaga

Asesores:

Dr. José Netz Romero Durán

Mtra. Núñez Reyes Alba Rocío

Ciudad de México, Agosto 2025

Índice

1. Introducción	4
1.1. Planteamiento del Problema	5
2. Objetivos	6
2.1. Objetivo General	6
2.2. Objetivos Específicos	6
3. Estado del Arte	7
3.1. PayPal	7
3.2. MEE6 Bot Discord	9
3.3. Tienda de Uniformes Escolares	12
3.4. Facebook Marketplace	14
3.5. AliExpress	15
4. Definición General del sistema	16
4.1. Propuesta	16
4.2. Tecnologías	17
4.2.1. PayPal	17
4.2.2. Discord	17
4.2.3. Node.js	18
4.2.4. React	18
4.2.5. MySQL	20
4.3. Requerimientos Funcionales	21
4.3.1. Página Web	21
4.3.2. Bot Discord	21
4.4. Requerimientos no Funcionales	22
4.4.1. Página Web	22
4.4.2. Bot Discord	22
5. Diseño del Sistema	23
5.1. Diagrama General	23
5.2. Diagramas de Secuencia	24
5.2.1. Diagramas de Secuencia de Autenticación y Compra	24
5.2.2. Diagramas de Secuencia de Notificación por Bot	25
5.3. Diagrama de la Base de Datos	26
5.3.1. Modelo Conceptual de la Base de Datos	26
5.3.2. Modelo Lógico de la Base de Datos	27
6. Desarrollo del sistema	28
6.1. APIs	28
6.1.1. API de PayPal	28
6.1.2. API de Discord OAuth2	32
6.2. Forma Física de la Base de datos	36
6.2.1. Tabla ticket	36
6.2.2. Tabla articulos_ticket	37
6.3. Filtro de productos	38
6.3.1. Arquitectura del Sistema de Filtros	38

6.3.2.	Filtro por Nombre de Producto	39
6.3.3.	Filtros Jerárquicos de Categorías	40
6.3.4.	Renderizado Optimizado con useEffect	41
6.3.5.	Función de Limpieza de Filtros	42
7.	Resultados	42
7.1.	Interfaces del Punto de Venta	42
7.1.1.	Interfaz de Cabecera para Login	42
7.1.2.	Interfaz de Cabecera con Usuario Autenticado	43
7.1.3.	Interfaz de Invitación a Discord	43
7.1.4.	Interfaz de Autenticación	44
7.1.5.	Interfaz del Carrito con Productos	46
7.1.6.	Interfaz del Carrito Vacío	46
7.1.7.	Interfaz de búsqueda de Productos	47
7.2.	Interfaces del Bot de Discord	47
7.2.1.	Creación de Comandos para Usuarios	48
7.2.2.	Comprobación de Tickets Pendientes	48
7.2.3.	Interfaz de Notificación vía Discord	49
7.2.4.	Envío de Notificaciones por Canales Privados	50
8.	Conclusiones	51
9.	Referencias	52

1. Introducción

En la actualidad, la digitalización de los procesos comerciales ha transformado la manera en que las empresas y los consumidores interactúan. Las plataformas de mensajería instantánea y redes sociales, como Discord, han dejado de ser simples herramientas de comunicación para convertirse en espacios donde se pueden gestionar comunidades, ofrecer soporte y hasta realizar transacciones comerciales. Este proyecto surge de la necesidad de aprovechar estas plataformas modernas para mejorar la experiencia de compra y la gestión de ventas en pequeños negocios y emprendimientos.

El objetivo principal de este trabajo es el desarrollo de un sistema de punto de venta (POS) en línea que se integra de manera nativa con un bot de Discord. Esta integración permite centralizar la comunicación entre clientes y vendedores, automatizar notificaciones sobre compras y suscripciones, y facilitar la gestión de pedidos en tiempo real. El sistema está diseñado para que los usuarios puedan autenticarse utilizando su cuenta de Discord, navegar por un catálogo de productos, agregar artículos a su carrito, realizar pagos seguros mediante PayPal y recibir notificaciones automáticas sobre sus compras directamente en Discord.

El desarrollo del proyecto abarca varias áreas tecnológicas: el frontend está construido con React, proporcionando una interfaz moderna, responsiva y fácil de usar; el backend utiliza Node.js y Express para gestionar la lógica de negocio y la comunicación con la base de datos MySQL, donde se almacena toda la información relevante de usuarios, productos, carritos y tickets de compra. La integración con Discord se realiza mediante la librería Discord.js, que permite crear un bot capaz de interactuar con los usuarios, responder a comandos y enviar notificaciones personalizadas. Además, se implementa la autenticación OAuth2 de Discord para garantizar la seguridad y simplicidad en el acceso de los usuarios.

Uno de los aspectos más importantes del sistema es la automatización de notificaciones: cada vez que un usuario realiza una compra, se genera un ticket en la base de datos con una bandera especial que indica que está pendiente de notificación. El bot de Discord revisa periódicamente estos tickets y, al encontrar nuevos, envía mensajes directos a los usuarios con los detalles de su compra, asegurando que la información llegue de manera rápida y directa, incluso si el usuario no revisa su correo electrónico.

El proyecto también aborda retos importantes, como la gestión segura de sesiones mediante cookies, la protección de datos sensibles, la integración de pagos electrónicos y el manejo de las limitaciones impuestas por la API de Discord en cuanto al envío de mensajes.

1.1. Planteamiento del Problema

En la actualidad se ha visto un crecimiento en la preferencia de las personas por las redes sociales y plataformas de mensajería instantánea como Discord¹, que combina comunidad, mensajería y redes sociales. Actualmente, muchas tiendas envían actualizaciones importantes únicamente por correo, lo que resulta en que los mensajes se ignoren o queden en carpetas de spam.

El correo electrónico genera fragmentación de la información y su efectividad ha disminuido considerablemente. Según Campaign Monitor, la tasa de apertura de correos comerciales es de apenas 21.33 % [6], mientras que las notificaciones por aplicaciones de mensajería como WhatsApp alcanzan tasas de apertura del 90 % [8]. Por esta razón, muchas empresas han comenzado a enviar notificaciones de compras y actualizaciones por redes sociales como WhatsApp, especialmente dirigidas a usuarios más jóvenes que prefieren estas plataformas.

Recientemente, las redes sociales han empezado a emplear mensajes automatizados o bots para agilizar las interacciones con los usuarios, evitando así que estos esperen a que una persona responda. Estos bots son software diseñado para realizar tareas específicas de forma repetida a una mayor velocidad que un ser humano; esto permite agilizar muchas tareas simples como enviar informes y/o mensajes.

Este proyecto planteó integrar un bot de Discord con una tienda en línea para centralizar la comunicación entre cliente y vendedor en la red social mencionada. Y así, abordar dos problemas clave: la dispersión de la comunicación y la falta de visibilidad de la información de compra.

El bot que se planteó para este proyecto pretende realizar notificaciones en tiempo real sobre pagos, actualizaciones de pedidos y recordatorios de suscripciones, facilitando la visibilidad de la compra.

¹Discord es una plataforma de comunicación diseñada para comunidades que permite mensajería de texto, voz y video. Disponible en: <https://discord.com>

2. Objetivos

En este apartado se presentan los objetivos planteados para guiar el desarrollo del proyecto y proponer una solución tecnológica para la gestión de compras y suscripciones mediante un bot de Discord.

2.1. Objetivo General

Desarrollar un sistema que permita el almacenamiento y consulta de los datos de compra de un punto de venta a través de un bot de Discord.

2.2. Objetivos Específicos

Para este proyecto se tienen los siguientes objetivos específicos.

- 1.- Configurar una base de datos que almacene y gestione la información de compras y suscripciones.
- 2.- Integrar un bot de Discord que permita realizar consultas a la base de datos.
- 3.- Desarrollar un sistema de notificaciones automatizadas.
- 4.- Evaluar las funcionalidades del sistema.
- 5.- Implementar un punto de venta.

3. Estado del Arte

Analizaremos sitios que cuenten con características o funcionalidades similares a las que usaremos para el proyecto que se va a desarrollar: las herramientas a analizar serán PayPal (forma de pago), bot de Discord MEE6, proyecto Tienda Virtual, Uniformes Escolares y Facebook Market.

3.1. PayPal

El método de pago de PayPal se basa principalmente en una implementación a través de su API, más que en el diseño de una interfaz propia. Esta API requiere información como la solicitud de pago, el precio del producto y la cuenta de PayPal a la que se debe depositar el dinero una vez que se complete la transacción. Por lo tanto, la única tarea que debe implementar el programador es la integración de la API y el botón para invocar el proceso de pago de PayPal.

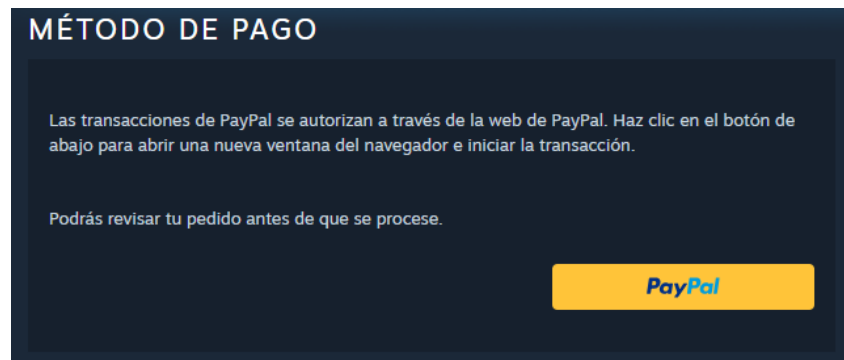


Figura 1: Botón de método de pago PayPal. [1]

El botón de la figura 1 no es más que una conexión de la página web, la sección de pago con el verdadero gestor de la transacción, como es PayPal, mediante la API.

Una vez llamada la API de PayPal, esta nos llevará a una ventana emergente (figura 2) donde se nos pedirá ingresar los datos de la cuenta de la plataforma PayPal para poder continuar con el cobro del servicio. Esta interfaz ya no es necesario crearla.

Inicie sesión en su cuenta de PayPal - Personal: Microsoft Edge

https://www.paypal.com/checkoutnow?atomic-event-state=eyJkb21haW4...



Pagar con PayPal

Ingrese su dirección de correo electrónico para empezar.

Correo electrónico o número de celular

[¿Olvidó su correo electrónico?](#)

Siguiente

o

Abrir una cuenta

[Cancelar y volver a www.steampowered.com](#)

[Español](#) | [English](#)

[Contactar](#) [Privacidad](#) [Acuerdos legales](#) [En todo el mundo](#)

Figura 2: Formulario de inicio de sesión de PayPal. [1]

La figura 2 muestra la interfaz para la autenticación desde la API de PayPal, donde el usuario debe autenticarse con su cuenta de PayPal para acceder al sistema. Se utiliza el sistema OAuth2 de PayPal para este propósito.

El pago entraría casi de inmediato, solo faltaría confirmar el pago para que se realice el cobro del servicio y finalizaría así el cobro por vía PayPal.

3.2. MEE6 Bot Discord

El bot MEE6 cuenta con muchas características interesantes para nosotros, pero en este proyecto nos centraremos en la posibilidad de enviar notificaciones de recordatorios por un canal específico o por mensaje directo al usuario.

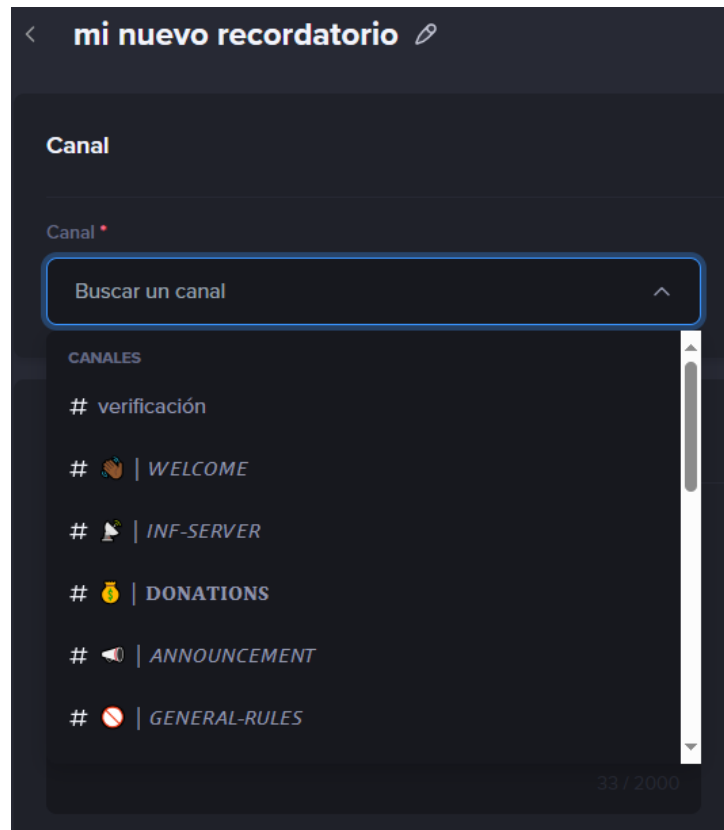


Figura 3: Selección de canal para recordatorio. [2]

En la ventana de la figura 3 podremos seleccionar en qué canal queremos que se genere el recordatorio que vamos a colocar. Esto es útil para aplicar recordatorios para un grupo de personas con roles específicos dentro del servidor de Discord. De esta forma, solo llegarán los recordatorios a las personas que queremos que reciban estos mensajes. También podemos poner un auto rol para que ellos decidan si quieren recibir los recordatorios o no, algo similar a cuando te pregunta una empresa si quieres recibir correos de promociones.

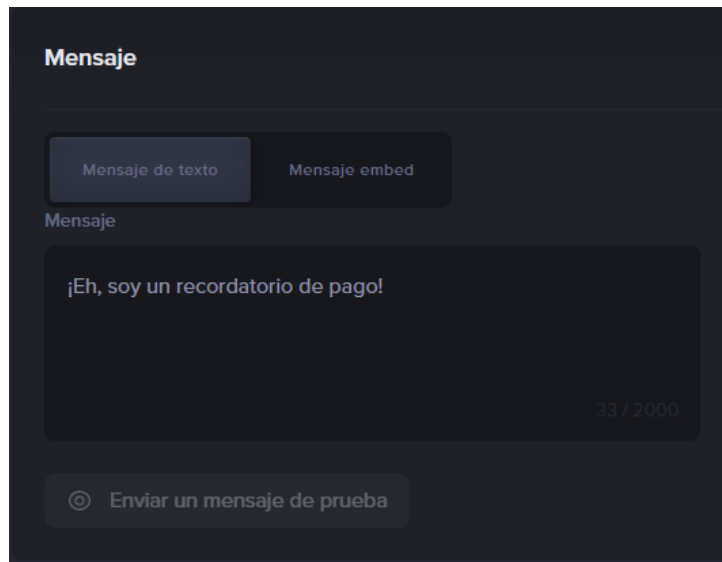


Figura 4: Ingreso del mensaje de recordatorio. [2]

La interfaz de la figura 4 permite ingresar el mensaje que deseamos que los usuarios reciban como recordatorio. En este caso, el mensaje consistirá únicamente en texto, aunque también es posible agregar una imagen para que el bot la publique automáticamente junto con el texto. De esta forma, se puede generar una publicidad mucho más eficaz para los artículos y el público objetivo al que va dirigida.

Además, al tener la posibilidad de enviar mensajes directos a los usuarios, podremos notificarles sobre próximas fechas de pago para alguna suscripción activa, o incluso ofrecerles descuentos personalizados.

Recordatorio

Único Múltiples

A las 18:00 ▼

Zona horaria UTC ⓘ

Día de la semana

✓ Mon ✓ Tue ✓ Wed ✓ Thu ✓ Fri ✓ Sat ✓ Sun

Figura 5: Configuración de recordatorios automáticos. [2]

La función de recordatorio se puede configurar de forma personalizada, como se muestra en la figura 5. Para que el recordatorio se genere automáticamente a intervalos de tiempo específicos, es necesario indicar el día y la hora. Esta funcionalidad es útil para evitar tener que crear el recordatorio de forma manual cada vez que se desee generar uno similar. En nuestro caso, tomará esta información de una base de datos, donde verificará si es necesario aplicar el recordatorio. En caso de tratarse de una mensualidad, recordará al usuario con 5 días de anticipación, revisando la fecha del siguiente cobro.

3.3. Tienda de Uniformes Escolares

En este proyecto desarrollado en la Universidad Abierta de Cataluña se realizó una tienda online de uniformes escolares, del cual tomaremos algunas utilidades de este proyecto como referencia.



Figura 6: Listado de artículos de la tienda de uniformes. [3]

Como podemos ver en la figura 6, en la interfaz de compra únicamente hace falta dar clic sobre 'añadir al carrito' para poder adquirirlo; también se ve el precio del producto y su nombre.

Del lado izquierdo también se puede apreciar el filtro que se tiene para los productos, de esta forma se tiene una mejor organización sobre lo que está observando el usuario comprador en este momento. Al lado derecho se muestran los productos que han sido añadidos al carrito, así como el precio final de la compra.

En la figura 7 podemos ver qué pasa al darle clic a comprar, ahora nos manda a la página de pago donde saldrán los datos del comprador, en este caso se ve cómo el pago se carga automáticamente a la mensualidad del colegio, en nuestro caso habrá un método de pago para realizar el pago.



Uniforme Escolar

Categorías

- ABRIGO
- CALZADO
- DEPORTIVO
- NIÑA
- NIÑO

 Mis pedidos

Detalle de pedido

Nombre: Pruebas
Dirección: Calle de Pruebas
Teléfono: 916458752
Email: correo@uoc.edu
NIF: 53038956C
CCC: 123456789123456 (*)

(*) El importe total se cargará en la cuenta indicada junto con el próximo recibo mensual del colegio

Unidades.	Artículo	Precio
1	ABRIGO UNIFORME	59,9€
1	CALCETIN	5,95€
1	PANTALON CORTO SPORT	19,9€
Total:		85,75€

<< Atrás Confirmar >>

Figura 7: Resumen de compra y datos del comprador. [3]

Para nuestro proyecto será importante colocar una sección de productos para seleccionarlos y, después de seleccionarlos, colocar una descripción y más imágenes para que el comprador pueda ver y estar más seguro de que se está comprando, al igual que un desglose de su compra total.

3.4. Facebook Marketplace

Facebook Marketplace también es otra de las plataformas más utilizadas para la compra y venta de productos, por esta razón es una buena referencia para el proyecto.

En la figura 8 podemos ver cómo se muestra la información del vendedor, detalles del producto y algunas especificaciones básicas; no es tan extensa la descripción como en Amazon. Sin embargo, nos da la posibilidad de enviarle un mensaje al comprador para poder preguntarle a él de una forma más directa y personalizada.

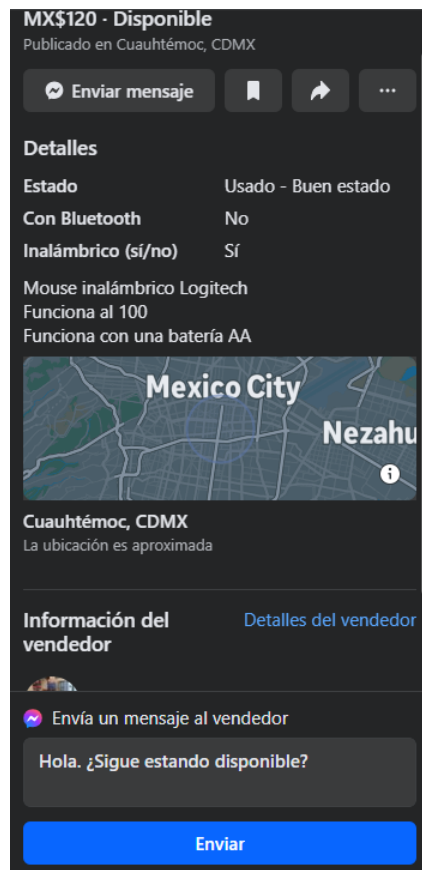


Figura 8: Información del producto y vendedor en Marketplace. [4]

La figura 8 muestra la sección informativa donde se puede ver el detalle del producto, así como la opción de contactar al vendedor.

Si bien esta plataforma está orientada a que terceros vendan sus productos en su tienda, son buenas referencias para este proyecto, al poder tener un trato directo con el vendedor.

Este proyecto tomará la idea de poder mandarle mensajes directamente al vendedor, en este caso, el administrador de la tienda, para algunas compras que requieran de más pasos y no solo vender el producto.

3.5. AliExpress

Algunas tiendas en línea ya utilizan actualmente notificaciones por medio de alguna red social; este es el caso de AliExpress, donde reenvían información básica de tu compra y links para visualizar la información completa de la compra en una página web. Si bien no es el ticket, sí facilita poder verlo sin necesidad de entrar al correo electrónico.

La figura 9 ejemplifica una notificación simplificada que recibiría el usuario tras realizar una compra.

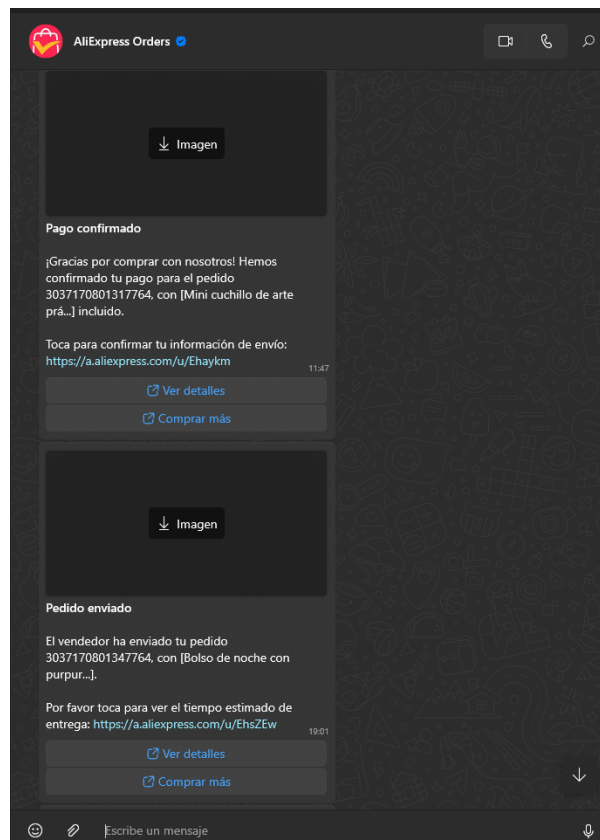


Figura 9: Notificación de compra reenviada (AliExpress). [5]

Este proyecto busca poder realizar notificaciones mediante una red social como es Discord para centralizar la forma de encontrar la información de compra y contactar con la tienda o persona encargada de esta de una forma mucho más fácil, utilizando únicamente Discord.

4. Definición General del sistema

En esta sección se describe la arquitectura general del sistema propuesto, así como las tecnologías a utilizar durante el desarrollo del sistema.

4.1. Propuesta

Este proyecto propone la creación de un punto de venta en línea integrado con un bot de Discord, con el fin de centralizar la comunicación y optimizar la gestión de compras y suscripciones. La propuesta incluye la implementación de métodos de pago seguros como PayPal, permitiendo a los usuarios realizar transacciones de manera confiable y eficiente.

El sistema busca aprovechar la versatilidad y popularidad de Discord como plataforma principal para ofrecer notificaciones en tiempo real sobre pagos, actualizaciones de pedidos y recordatorios de suscripciones. La integración del bot con la tienda en línea permite automatizar completamente las notificaciones de tickets de compra y suscripciones adquiridas, las cuales son enviadas directamente vía Discord a los usuarios correspondientes.

El proyecto incluye el diseño y desarrollo de una base de datos robusta para almacenar y consultar información relevante como datos de usuarios, catálogo de productos, historial de compras, tickets generados y estados de notificación. Esta base de datos soporta la integración del bot para realizar consultas y notificaciones automatizadas.

También se contempla la implementación de autenticación mediante OAuth2 de Discord, eliminando la necesidad de crear cuentas adicionales y aprovechando la infraestructura existente de la plataforma. El sistema de carritos de compra permite a los usuarios agregar productos, modificar cantidades y proceder al pago de manera intuitiva.

Además, se incluye un sistema de notificaciones inteligente que verifica periódicamente nuevos tickets de compra y suscripciones, enviando mensajes personalizados a través del bot de Discord. Esto mejora significativamente la visibilidad de la información de compra y la interacción directa con los usuarios.

4.2. Tecnologías

Las tecnologías para este proyecto serán API PayPal, API Discord, MySQL, Node.js, React. Se explicará cada una de las tecnologías para definir qué funcionalidad van a desempeñar en el proyecto propuesto.

4.2.1. PayPal

PayPal es una de las plataformas más populares para pagos en línea, conocida por su facilidad de uso, seguridad y aceptación internacional. Para este proyecto, se ha elegido PayPal debido a su amplia adopción por los usuarios y su capacidad para gestionar pagos transfronterizos de manera eficiente, ofreciendo una ventaja frente a competidores como Mercado Pago.

La integración de PayPal en una tienda en línea se realiza principalmente mediante su API. Esta API permite establecer la comunicación entre la tienda y los servidores de PayPal para gestionar las transacciones de manera segura.

Una ventaja adicional de PayPal es que proporciona un entorno de pruebas (sandbox) que facilita el desarrollo y testing del sistema. Este entorno permite realizar transacciones simuladas sin manejar dinero real, lo que es ideal para verificar que la integración funciona correctamente y que los mensajes de confirmación de compra se reciben adecuadamente a través de la API. Esto garantiza que el sistema esté completamente funcional antes de implementarlo en producción.

4.2.2. Discord

Discord es una plataforma de comunicación ampliamente utilizada para la creación y gestión de comunidades temáticas. Originalmente orientada hacia los jugadores, Discord ha evolucionado para ser una herramienta versátil que permite administrar grandes grupos de usuarios gracias a su estructura basada en servidores. En estos servidores, se pueden asignar roles, gestionar permisos y utilizar bots para automatizar diversas tareas, convirtiéndola en una solución ideal para centralizar la interacción entre usuarios y servicios.

Discord facilita significativamente el desarrollo de bots mediante su API robusta y bibliotecas especializadas como Discord.js para Node.js. Estas herramientas simplifican la implementación de funcionalidades avanzadas como comandos slash, embeds interactivos, gestión de eventos en tiempo real y notificaciones automáticas.

Discord será una herramienta central en este proyecto, gracias a su capacidad para gestionar usuarios y automatizar notificaciones. Se aprovecharán tanto su API como las características de inicio de sesión para integrar la tienda en línea con la plataforma.

Se utilizará la OAuth2 API de Discord para implementar un sistema de autenticación seguro y sencillo. Esto permitirá a los usuarios iniciar sesión en la tienda en línea con su cuenta de Discord, sin necesidad de crear un nuevo usuario.

4.2.3. Node.js

Node.js es un entorno de ejecución que permite ejecutar JavaScript fuera del navegador. Basado en el motor V8 de Google Chrome, Node.js es extremadamente rápido y eficiente, especialmente para aplicaciones que requieren manejo en tiempo real y procesamiento asíncrono.

Una de las principales ventajas de Node.js es su capacidad para funcionar como servidor web completo, reemplazando la necesidad de servidores tradicionales como Apache o Nginx en muchos casos. Node.js incluye un servidor HTTP integrado que permite crear aplicaciones web desde cero sin depender de software adicional. Esto facilita significativamente el desarrollo y despliegue de aplicaciones, especialmente en entornos locales donde se puede levantar un servidor completo.

Node.js es ideal para aplicaciones que necesitan manejar múltiples solicitudes simultáneamente mediante su arquitectura de eventos no bloqueante. Esta característica lo convierte en la opción perfecta para aplicaciones como bots de Discord que deben responder a eventos en tiempo real, procesar múltiples comandos de usuarios y mantener conexiones persistentes. Su estabilidad permite conservar un buen rendimiento incluso con muchas solicitudes simultáneas.

Para este proyecto, Node.js actúa como servidor backend para la tienda en línea, gestionando la lógica de negocio, autenticación y comunicación con la base de datos; y además sirve como plataforma para el bot de Discord mediante la biblioteca Discord.js.

Discord.js

Discord.js es una biblioteca de Node.js específicamente diseñada para simplificar la interacción con la API de Discord. Esta librería proporciona una abstracción de alto nivel que facilita enormemente el desarrollo de bots, eliminando la complejidad de manejar directamente las conexiones WebSocket y las peticiones HTTP a la API de Discord.

Discord.js ofrece características avanzadas como manejo automático de reconexiones, sistema de eventos robusto, tipado para TypeScript, y métodos simplificados para todas las funcionalidades de Discord. Permite implementar fácilmente comandos slash, embeds interactivos con botones y menús desplegable, gestión de roles y permisos, y notificaciones automáticas.

En este proyecto, Discord.js permite que el bot pueda registrar comandos dinámicamente, enviar mensajes directos personalizados a los usuarios tras completar una compra y realizar una gestión de eventos en tiempo real como la comprobación de los nuevos tickets creados en la Base de Datos.

4.2.4. React

React es una biblioteca de JavaScript desarrollada por Facebook, diseñada para construir interfaces de usuario (UI) modernas, interactivas y dinámicas. Se enfoca principalmente en la creación de aplicaciones a través de componentes reutilizables, lo que

permite dividir la interfaz en pequeñas piezas independientes y manejables. Esto mejora la organización y el mantenimiento del código; también facilita la reutilización de estos componentes en diferentes partes de la aplicación. En este proyecto, un componente podría encargarse de mostrar los productos de la tienda, otro del carrito de compras y otro de las notificaciones de compra, optimizando así el desarrollo.

Una de las características más destacadas de React es el uso del Virtual DOM. Este mecanismo crea una representación en memoria del DOM real, lo que permite que React identifique y actualice solo las partes necesarias de la interfaz en lugar de renderizarla por completo cada vez que hay un cambio. Esto mejora significativamente el rendimiento de la aplicación, especialmente en escenarios donde las actualizaciones en tiempo real son frecuentes, como al agregar productos al carrito o actualizar el estado de un pedido.

React fue elegido para este proyecto debido a su escalabilidad y facilidad de mantenimiento. React ofrece un equilibrio ideal entre flexibilidad y robustez, lo que lo convierte en una herramienta esencial para construir una tienda en línea moderna que integre funcionalidades avanzadas como notificaciones en tiempo real y bots automatizados.

4.2.5. MySQL

MySQL es un sistema de gestión de bases de datos relacional, se utiliza para almacenar, organizar y administrar grandes volúmenes de información en aplicaciones de software. Este sistema utiliza el lenguaje SQL (Structured Query Language) para interactuar con los datos, permitiendo realizar operaciones como consultas, inserciones, actualizaciones y eliminaciones de manera eficiente y estructurada.

Una de las principales ventajas de MySQL es su capacidad para manejar grandes cantidades de datos de manera rápida y eficiente. Gracias a su diseño optimizado, MySQL permite realizar consultas complejas que filtran y procesan información específica en cuestión de milisegundos. Esto es crucial en aplicaciones como tiendas en línea, donde se manejan continuamente datos relacionados con usuarios, productos, pedidos y transacciones.

Otra característica importante de MySQL es su enfoque en la integridad de los datos y la seguridad. A través de herramientas como Prepared Statements (consultas preparadas), es posible proteger la base de datos contra ataques comunes como la inyección SQL, garantizando que las consultas se ejecuten de forma segura y controlada. Esta funcionalidad es esencial en proyectos que manejan información confidencial, como detalles de usuarios y transacciones económicas.

En este proyecto, MySQL se utilizará para gestionar tablas que almacenen información esencial como detalles de los usuarios, productos, pedidos y estados de pago. Además, permitirá realizar operaciones frecuentes, como buscar productos según filtros personalizados, actualizar el estado de un pedido en real tiempo o registrar nuevas transacciones.

4.3. Requerimientos Funcionales

A continuación, se presentan los requerimientos funcionales del proyecto, los cuales describen las acciones y comportamientos que el sistema deberá cumplir para satisfacer las necesidades del usuario y alcanzar los objetivos planteados. Estos requerimientos definen qué funciones específicas realizará el sistema, incluyendo la interacción con el bot de Discord, la gestión de pedidos y la comunicación con los clientes.

4.3.1. Página Web

- Permitir a los usuarios navegar por los productos disponibles en la tienda virtual.
- Permitir a los usuarios autenticarse mediante Discord (OAuth2).
- Permitir a los usuarios agregar productos al carrito de compras.
- Permitir a los usuarios modificar la cantidad de productos en el carrito y eliminar productos del mismo.
- Permitir a los usuarios realizar compras y generar tickets de compra usando PayPal.
- Mostrar notificaciones de éxito o error en las operaciones principales (login, agregar al carrito, compra).
- Permitir a los usuarios ver su perfil y cerrar sesión.
- Mostrar información relevante de los productos (nombre, imagen, precio, categoría).
- Permitir la navegación entre diferentes páginas (productos, carrito, perfil).
- Integración con un servidor de Discord (botón de invitación y QR).
- Permitir que los usuarios puedan realizar pagos electrónicos por PayPal.

4.3.2. Bot Discord

- Permitir la integración con el servidor de Discord de la tienda virtual.
- Responder a comandos básicos de los usuarios /productos.
- Notificar en canales específicos cuando se realiza una compra o se genera un ticket en la tienda.
- Permitir la gestión de roles o permisos para usuarios que compran productos.
- Permitir la consulta de información de productos o tickets desde Discord (si aplica).

4.4. Requerimientos no Funcionales

En esta sección se detallan los requerimientos no funcionales del proyecto, los cuales establecen los criterios de calidad que el sistema debe cumplir.

4.4.1. Página Web

- La aplicación debe ser responsiva y funcionar correctamente en dispositivos móviles y de escritorio.
- El sistema debe mantener la sesión del usuario mediante cookies seguras.
- El backend debe manejar la autenticación de manera segura.
- El sistema debe manejar errores y mostrar mensajes claros al usuario.
- El sistema debe ser escalable para permitir la integración de nuevas funcionalidades (por ejemplo, más métodos de pago, más categorías).
- El sistema debe proteger los datos sensibles y cumplir con buenas prácticas de seguridad.
- El backend debe soportar concurrencia y múltiples usuarios simultáneos.
- El sistema debe permitir la integración con servicios externos como Discord y PayPal.

4.4.2. Bot Discord

- El bot debe estar disponible 24/7 y manejar desconexiones de manera automática.
- El código debe estar organizado y documentado para facilitar el mantenimiento.
- El bot debe manejar errores y excepciones sin interrumpir su funcionamiento.
- El bot debe proteger los tokens y credenciales de acceso.
- El bot debe ser escalable para permitir la adición de nuevos comandos o integraciones.

5. Diseño del Sistema

En esta sección se presenta el diseño del sistema, incluyendo diagramas que ilustran la arquitectura, los flujos de interacción y la estructura de la base de datos. Estos diagramas proporcionan una visión clara de cómo se integran los diferentes componentes del sistema y cómo interactúan entre sí para cumplir con los requerimientos funcionales y no funcionales establecidos.

5.1. Diagrama General

El diagrama general del sistema muestra los principales componentes y su interacción. A continuación se describen los elementos clave:

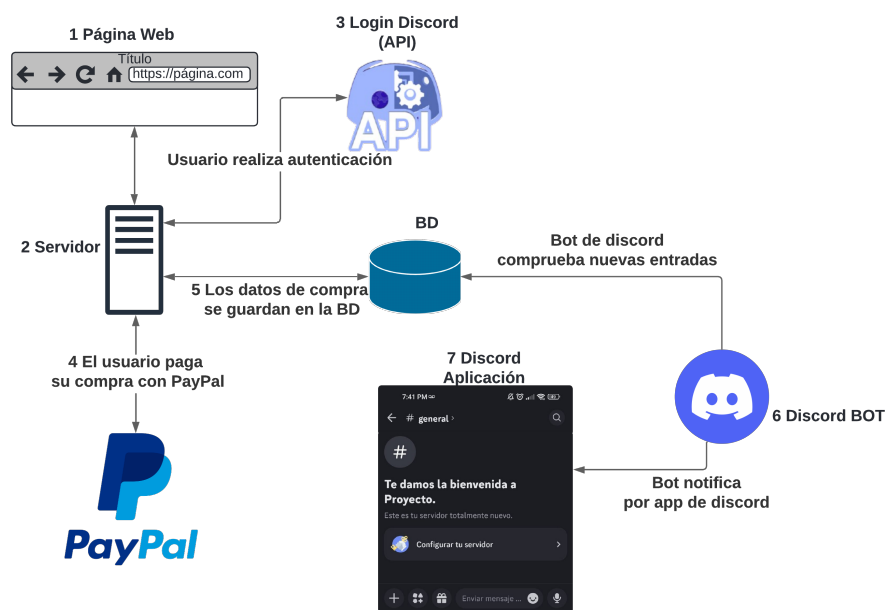


Figura 10: Diagrama general del sistema

- 1.- El usuario podrá consultar los artículos a la venta en el sitio web.
- 2.- El servidor será el encargado de procesar todas las solicitudes que conectarán el front-end con toda la lógica.
- 3.- El sistema autenticará al usuario mediante su cuenta de Discord.
- 4.- Una vez teniendo artículos en el carrito de compra, el usuario podrá comprarlos y pagarlos mediante PayPal; deberá realizar una autenticación dentro de PayPal para poder acceder a su cuenta y realizar el pago.
- 5.- En la base de datos se guardarán todos los datos necesarios para autenticación, los datos de compra, y los artículos que se han colocado a la venta.

6.- El bot de Discord hará una función administrativa, esta será avisarle al administrador y al usuario comprador sobre su compra o próximas suscripciones. Estos avisos serán personales para cada usuario.

7.- Mediante Discord, el usuario recibirá las notificaciones que le enviará el bot; el usuario requiere un perfil dentro de Discord.

5.2. Diagramas de Secuencia

En esta sección se presentan los diagramas de secuencia que ilustran los flujos de interacción entre los diferentes objetos durante los procesos de autenticación y compra.

5.2.1. Diagramas de Secuencia de Autenticación y Compra

El diagrama secuencial de autenticación y compra muestra el envío de mensajes e interacción entre los objetos durante el proceso de autenticación del usuario y la realización de una compra.

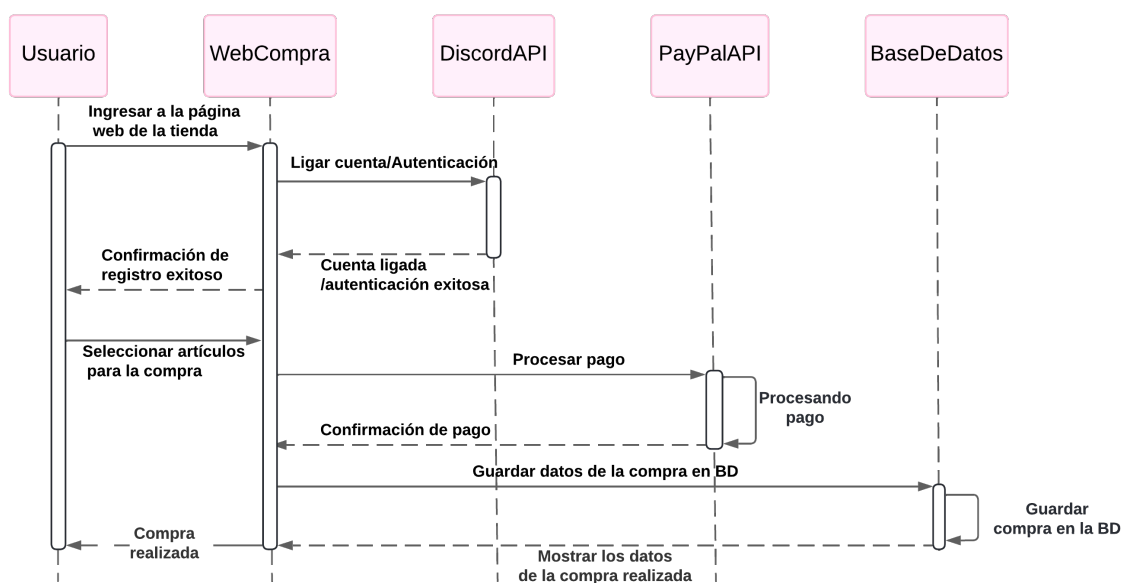


Figura 11: Diagrama secuencial de autenticación y compra

1.- El usuario deberá iniciar sesión, autenticarse en la página web para poder realizar compras.

2.- Una vez seleccionado el artículo de compra, se mandará al usuario a realizar su compra; se usará PayPal para realizar la validación de compra.

3.- La compra realizada se registrará en la base de datos.

4.- Se le mostrará al usuario un mensaje de compra exitosa.

5.2.2. Diagramas de Secuencia de Notificación por Bot

El diagrama secuencial de notificación por bot muestra el flujo de interacción entre los objetos durante el proceso de notificación al usuario sobre su compra.

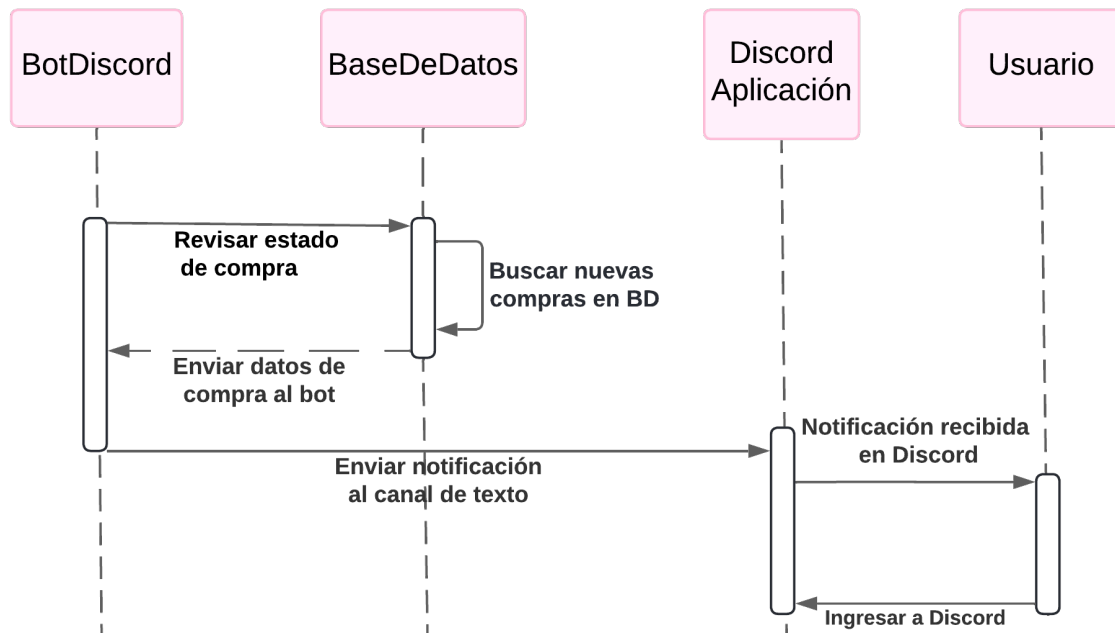


Figura 12: Diagrama secuencial de notificación por bot

- 1.- El bot de Discord comprobará si hay nuevas entradas en la BD.
- 2.- El bot enviará los datos de compra vía Discord.
- 3.- El usuario comprador podrá ver los datos de su compra en el mensaje dejado por el bot.

5.3. Diagrama de la Base de Datos

En esta sección se presenta el diseño de la base de datos, incluyendo su modelo conceptual y lógico.

5.3.1. Modelo Conceptual de la Base de Datos

Se presentará el modelo conceptual de la base de datos, que incluye las entidades principales y sus relaciones.

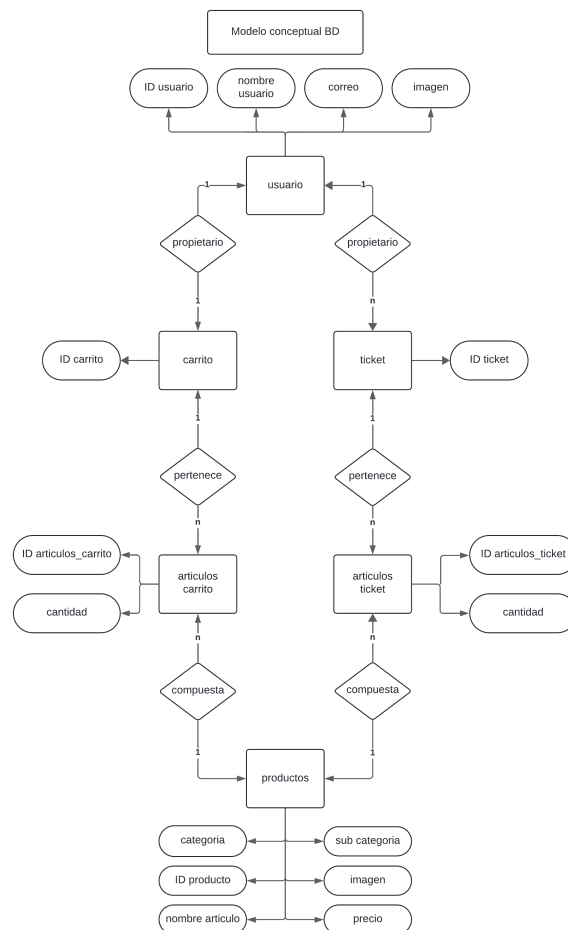


Figura 13: Modelo conceptual de la base de datos

El diagrama de la figura 13 comienza con la entidad usuario, que contiene información básica como ID, nombre, correo e imagen. Cada usuario puede ser propietario de un carrito y puede tener muchos tickets, lo que implica que un mismo usuario puede realizar varias compras, pero solo tener un carrito activo a la vez.

El carrito está relacionado con el usuario mediante una relación de propiedad uno a uno, y contiene múltiples artículos del carrito, es decir, productos que el usuario ha agregado antes de comprar. Estos artículos tienen una cantidad y se relacionan con la entidad productos, lo cual indica qué producto específico se añadió y en qué cantidad.

Una vez que el usuario realiza una compra, se genera un ticket, también asociado al usuario, pero en una relación de uno a muchos (un usuario puede tener muchos tickets).

El ticket contiene artículos del ticket, similares a los del carrito, con su respectiva cantidad y referencia al producto adquirido.

La entidad productos es el catálogo general, donde se guarda información de cada artículo como su ID, nombre, categoría, subcategoría, imagen y precio. Esta entidad es común tanto para los artículos del carrito como para los del ticket, ya que los productos son los mismos, solo cambian de estado según estén en proceso de compra (carrito) o ya comprados (ticket).

5.3.2. Modelo Lógico de la Base de Datos

Se presentará el modelo lógico de la base de datos, que incluye las tablas y sus relaciones así como sus atributos.

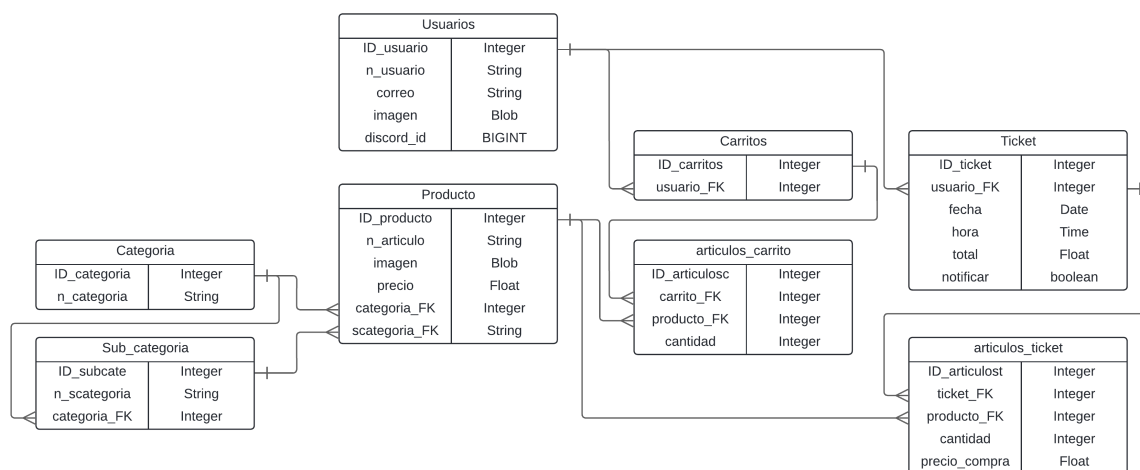


Figura 14: Diagrama lógico de la base de datos

En el diagrama de la figura 14 de base de datos, la entidad Usuarios almacena la información del cliente como ID, nombre, correo e imagen. Cada usuario puede tener un carrito (relación uno a uno) y también puede generar tickets de compra (relación uno a muchos).

El carrito está relacionado con el usuario mediante la clave foránea usuario_FK y contiene productos a través de la tabla articulos_carrito, donde se guarda qué producto fue agregado producto_FK y en qué cantidad.

Cuando se realiza una compra, se genera un ticket que también pertenece a un usuario (usuario_FK) y contiene información como fecha, hora, total de la compra y si se notificó. Los productos comprados se guardan en la tabla articulos_ticket, con su cantidad y el precio al momento de la compra (precio_compra), todo enlazado al ticket (ticket_FK) y al producto (producto_FK).

La entidad Producto almacena todos los artículos disponibles: su ID, nombre, imagen, precio y dos claves foráneas que lo conectan con su categoría y subcategoría (categoria_FK

y `scategoria_FK`). Las categorías están organizadas en dos niveles, mediante las tablas `Categoria` y `Sub_categoria`, esta última relacionada con la primera a través de `categoria_FK`.

6. Desarrollo del sistema

En esta sección se describen las APIs que se implementarán en el sistema, tanto internas como externas. Estas APIs son fundamentales para la comunicación entre los diferentes componentes del sistema, como la tienda en línea y el bot de Discord, facilitando el intercambio de datos de forma estructurada y segura. A través de estas interfaces, el sistema podrá recibir eventos, enviar notificaciones y ejecutar funcionalidades clave que permiten la integración fluida entre la plataforma de e-commerce y Discord.

6.1. APIs

En esta sección se describen las APIs que formarán parte del sistema, tanto internas como externas. Las APIs (Interfaces de Programación de Aplicaciones) permiten la comunicación entre los diferentes componentes del sistema, como la tienda en línea y el bot de Discord, facilitando el intercambio de datos de forma estructurada y segura. A través de estas interfaces, el sistema podrá recibir eventos, enviar notificaciones y ejecutar funcionalidades clave que permiten la integración fluida entre la plataforma de e-commerce y Discord.

6.1.1. API de PayPal

PayPal es la plataforma de pagos elegida para este proyecto debido a su facilidad de implementación, seguridad y amplia aceptación entre los usuarios. La integración se realizó utilizando el SDK oficial `@paypal/checkout-server-sdk` para Node.js, que proporciona métodos seguros y eficientes para procesar transacciones.

Configuración del Entorno PayPal

La configuración se realizó creando un cliente PayPal dedicado que maneja la comunicación con la API. Se utilizó el entorno sandbox para desarrollo y pruebas, configurado directamente en el código:

Código 1: Configuración del cliente PayPal

```
1 // Cliente PayPal SDK para procesar pagos online
2 const paypal = require('@paypal/checkout-server-sdk');
3
4 // Configuración del entorno PayPal en modo sandbox (desarrollo/pruebas
5 // )
6 const environment = new paypal.core.SandboxEnvironment(
7   // Client ID de PayPal para aplicacion sandbox
8   'AW4xEijNfSBJiSYtmQn0f_ABCdasd1-dNdjmw_XZ2fIDPrbvqZQZBSy4fQn',
9   // Client Secret de PayPal (CONFIDENCIAL - para autenticación del
10  servidor)
11  'EHImBnGza7Csas8CsdscaKHfbHV9s8SN0k39YW9L5A9B_fISZwPPP'
12 );
13
14 // Cliente HTTP configurado para comunicarse con la API de PayPal
15 const client = new paypal.core.PayPalHttpClient(environment);
16
17 module.exports = client;
```

Implementación del Flujo de Pago

El proceso de pago se divide en dos etapas principales: la creación de la orden en el frontend y la captura del pago en el backend. Este flujo garantiza la seguridad de la transacción y permite verificar el estado del pago antes de procesar la compra.

Etapas 1: Creación de la Orden (Frontend)

En el frontend, se utiliza el componente PayPalButtons de la librería @paypal/react-paypal-js. Este componente maneja automáticamente la interfaz de usuario y la comunicación inicial con PayPal:

Código 2: Implementación del botón PayPal en React

```
1 // Configuración inicial del SDK de PayPal
2 const initialOptions = {
3   "client-id": "AW4xEijNfSBJiSYtmQnOf_ABCdasd1-
4     dNdjmw_XZ2fIDPrbvqZQZBSy4fQn",
5   currency: "MXN", // Moneda mexicana
6 };
7
8 <PayPalButtons
9   style={{ layout: "vertical" }}
10   // Función que se ejecuta al crear la orden de pago
11   createOrder={(data, actions) => {
12     return actions.order.create({
13       purchase_units: [
14         {
15           amount: {
16             value: total.toFixed(2),
17           },
18         },
19       ],
20     });
21   }}
22   // Función que se ejecuta cuando PayPal aprueba el pago
23   onApprove={async (data, actions) => {
24     try {
25       const totalFloat = parseFloat(total);
26
27       // Enviar datos de la orden al backend para completar el
28       // pago
29       const response = await fetch(`${process.env.
30         REACT_APP_BACKEND_URL}/api/paypal/capture-order`, {
31         method: 'POST',
32         headers: {
33           'Content-Type': 'application/json',
34         },
35         credentials: 'include', // Incluir cookies de
36         // autenticación
37         body: JSON.stringify({ orderId: data.orderID, total:
38           totalFloat }),
39       });
40
41       if (response.ok) {
42         // Pago exitoso - mostrar mensaje y recargar pagina
43         setPopupMessage('Pago completado exitosamente');
44         window.location.reload();
45       }
46     } catch (error) {
47       console.error('Error al capturar el pago:', error);
48     }
49   }}
50 />
```

Etapas 2: Captura del Pago (Backend)

Una vez que el usuario aprueba el pago en la interfaz de PayPal, el frontend envía el ID de la orden al backend para completar la captura usando el SDK de PayPal:

Código 3: Captura de orden PayPal en el backend

```
1 // Ruta para capturar una orden de PayPal y procesar la compra
2 router.post('/capture-order', async (req, res) => {
3   const { orderId, total } = req.body;
4
5   // Crear solicitud de captura usando el SDK de PayPal
6   const request = new paypal.orders.OrdersCaptureRequest(orderId);
7   request.requestBody({}); // Cuerpo vacío para captura simple
8
9   try {
10    // Ejecutar la captura de la orden en PayPal usando su API
11    const capture = await client.execute(request);
12    const order = capture.result;
13
14    // Verificar que el pago se completo exitosamente en PayPal
15    if (order.status === 'COMPLETED') {
16      // Obtener información del usuario desde las cookies
17      const userCookie = req.cookies.user;
18      const user = JSON.parse(userCookie);
19      const discordId = user.id;
20
21      // Buscar el ID del usuario en la base de datos
22      const [rows] = await pool.query('SELECT ID_usuarios FROM
23        usuarios WHERE discord_id = ?', [discordId]);
24      const userId = rows[0].ID_usuarios;
25
26      // Crear el ticket de compra usando el modelo Ticket
27      const ticketId = await Ticket.create(userId, total);
28
29      res.json({ success: true, message: 'Pago completado
30        exitosamente y ticket creado.', ticketId });
31    } else {
32      res.status(400).json({ success: false, message: 'El pago no
33        se completó.' });
34    }
35  } catch (error) {
36    console.error('Error al capturar el pago:', error);
37    res.status(500).json({ success: false, message: 'Error al
38      capturar el pago.' });
39  }
40 }
```

Este flujo de dos etapas permite que PayPal maneje toda la complejidad de la autenticación del usuario, la validación de métodos de pago y el procesamiento seguro de la transacción, mientras que el backend se encarga de confirmar y registrar la compra una vez completada. Una ventaja clave de PayPal es su entorno sandbox que permite realizar pruebas completas sin manejar dinero real, verificando que la integración funciona correctamente antes del despliegue en producción.

6.1.2. API de Discord OAuth2

La implementación de Discord OAuth2 permite a los usuarios autenticarse utilizando sus credenciales de Discord, eliminando la necesidad de crear cuentas adicionales.

Configuración de la Aplicación Discord

Para implementar OAuth2 con Discord, primero se registró una aplicación en el Portal de Desarrolladores de Discord. En el backend, se configuró un controlador de autenticación que centraliza todas las operaciones OAuth2:

Código 4: Configuración del controlador de autenticación Discord

```
1 class AuthController {
2   constructor() {
3     // Configuración de endpoints de Discord API
4     this.discordOAuth2URL = 'https://discord.com/api/oauth2/
      authorize';
5     this.tokenURL = 'https://discord.com/api/oauth2/token';
6     this.userInfoURL = 'https://discord.com/api/users/@me';
7     this.redirectURI = process.env.DISCORD_REDIRECT_URI;
8     this.clientID = process.env.DISCORD_CLIENT_ID;
9     this.clientSecret = process.env.DISCORD_CLIENT_SECRET;
10  }
11 }
```

Flujo de Autenticación OAuth2

El proceso OAuth2 de Discord sigue un flujo estándar de autorización que se divide en varias etapas claramente definidas:

Etapas 1: Inicio del Flujo de Autorización

Cuando el usuario hace clic en Iniciar sesión con Discord, el backend genera una URL de autorización y redirigirá al usuario a Discord:

Código 5: Inicio del flujo OAuth2

```
1 login(req, res) {
2   const scope = 'identify email'; // Permisos solicitados a Discord
3   const state = crypto.randomBytes(16).toString('hex'); // Token anti
      -CSRF
4
5   // Guardar estado en cookie temporal para validación posterior
6   res.cookie('oauth_state', state, {
7     httpOnly: true,
8     maxAge: 60000, // 1 minuto de expiración
9     sameSite: 'lax'
10  });
11
12  // Construir URL de autorización Discord
13  const url = `${this.discordOAuth2URL}?client_id=${this.clientID}&
      redirect_uri=${encodeURIComponent(this.redirectURI)}&
      response_type=code&scope=${encodeURIComponent(scope)}&state=${
      state}`;
14  res.redirect(url);
15 }
```

En esta etapa, el sistema genera un token 'state' aleatorio que se almacena temporalmente en una cookie para prevenir ataques CSRF (Cross-Site Request Forgery). Discord requiere los permisos 'identify' y 'email' para obtener información básica del usuario.

Etapa 2: Autorización del Usuario en Discord

Discord presenta al usuario una pantalla de autorización donde debe aprobar los permisos solicitados. Una vez autorizado, Discord redirige al usuario de vuelta a la aplicación con un código de autorización temporal.

La imagen de la interfaz se mostrará más adelante figura ??.

Etapa 3: Intercambio de Código por Token de Acceso

El callback del backend procesa la respuesta de Discord, valida el estado e intercambia el código de autorización por un token de acceso:

Código 6: Callback OAuth2 - Intercambio de token

```
1 async callback(req, res) {
2   const { code, state } = req.query;
3   const savedState = req.cookies.oauth_state;
4
5   // Verificar estado para prevenir ataques CSRF
6   if (!code || !state || state !== savedState) {
7     return res.status(400).redirect(`${process.env.FRONTEND_URL}/`)
8   }
9
10  try {
11    // Intercambiar codigo de autorizacion por token de acceso
12    const tokenResponse = await axios.post(this.tokenURL, new
13      URLSearchParams({
14        client_id: this.clientID,
15        client_secret: this.clientSecret,
16        grant_type: 'authorization_code',
17        code,
18        redirect_uri: this.redirectURI,
19      }), {
20        headers: {
21          'Content-Type': 'application/x-www-form-urlencoded',
22        },
23      });
24    const accessToken = tokenResponse.data.access_token;
```

Etapa 4: Obtención de Datos del Usuario

Con el token de acceso, el sistema consulta la API de Discord para obtener la información del usuario:

Código 7: Obtención de datos del usuario desde Discord

```
1 // Obtener informacion del usuario desde Discord
2 const userResponse = await axios.get(this.userInfoURL, {
3   headers: {
4     Authorization: 'Bearer ${accessToken}',
5   },
6 });
7
8 const userData = userResponse.data;
9
10 // Buscar o crear usuario en la base de datos local
11 const [rows] = await pool.query('SELECT * FROM usuarios WHERE
12   discord_id = ?', [userData.id]);
13
14 if (rows.length === 0) {
15   // Insertar nuevo usuario
16   await pool.query(
17     'INSERT INTO usuarios (discord_id, correo, imagen,
18       n_usuario) VALUES (?, ?, ?, ?)',
19     [userData.id, userData.email, userData.avatar, userData
20       .username]
21   );
22 } else {
23   // Actualizar datos del usuario existente
24   await pool.query(
25     'UPDATE usuarios SET correo = ?, imagen = ?, n_usuario
26       = ? WHERE discord_id = ?',
27     [userData.email, userData.avatar, userData.username,
28       userData.id]
29   );
30 }
```

Etapa 5: Creación de Sesión Mediante Cookie

Finalmente, el sistema crea una cookie de sesión segura que contiene los datos esenciales del usuario:

Código 8: Creación de cookie de sesión

```
1 // Preparar datos del usuario para la cookie de sesion
2 const userPayload = {
3   id: userData.id,
4   username: userData.global_name || userData.username,
5   // Construir URL de avatar usando Discord CDN
6   avatar: userData.avatar
7     ? `https://cdn.discordapp.com/avatars/${userData.id}/${userData.avatar}.png`
8     : `https://cdn.discordapp.com/embed/avatars/${userData.discriminator % 5}.png`,
9   email: userData.email,
10 };
11
12 // Crear cookie de sesion con datos del usuario
13 res.cookie('user', JSON.stringify(userPayload), {
14   httpOnly: true, // Solo accesible desde el servidor (seguridad)
15   secure: false, // Para desarrollo local (HTTPS en produccion)
16   maxAge: 24 * 60 * 60 * 1000, // 24 horas de duracion
17   sameSite: 'lax' // Proteccion contra CSRF
18 });
19
20 // Redirigir al frontend tras autenticacion exitosa
21 res.redirect(`${process.env.FRONTEND_URL}/`);
22 } catch (error) {
23   console.error('Error during Discord authentication:', error);
24   res.redirect(`${process.env.FRONTEND_URL}/`);
25 }
26 }
```

Gestión de Sesiones Mediante Cookies

El sistema utiliza cookies HTTP para mantener la sesión del usuario autenticado. La cookie 'user' contiene información esencial en formato JSON y se configura con las siguientes características de seguridad:

- **httpOnly**: La cookie solo es accesible desde el servidor, no desde JavaScript del cliente, protegiendo contra ataques XSS.
- **sameSite**: Configurada como 'lax' para protección básica contra CSRF.
- **maxAge**: Duración de 24 horas, después de las cuales el usuario debe autenticarse nuevamente.
- **secure**: Deshabilitado para desarrollo local, debe habilitarse en producción con HTTPS.

La estructura de la cookie de usuario contiene:

Código 9: Estructura de la cookie de sesión

```
1 {  
2   "id": "discord_user_id",           // ID unico del usuario en Discord  
3   "username": "nombre_usuario",     // Nombre de usuario o nombre  
    global  
4   "avatar": "https://cdn.discordapp.com/avatars/...", // URL del avatar  
5   "email": "usuario@email.com"      // Correo electronico del usuario  
6 }
```

Verificación de Sesión

Para operaciones posteriores, el sistema verifica la autenticación leyendo la cookie del usuario:

Código 10: Verificación de usuario autenticado

```
1 // Ruta para obtener informacion del usuario autenticado  
2 router.get('/user', (req, res) => {  
3   const userCookie = req.cookies.user;  
4   if (!userCookie) {  
5     return res.status(401).send('Not authenticated');  
6   }  
7  
8   try {  
9     const user = JSON.parse(userCookie);  
10    res.json(user); // Devolver datos del usuario autenticado  
11  } catch (error) {  
12    res.status(400).send('Invalid cookie format');  
13  }  
14 });
```

Implementar el inicio de sesión por medio de Discord nos permite evitar registro de usuarios y almacenamiento de contraseñas dejando la seguridad para comprobar la identidad del usuario a Discord.

6.2. Forma Física de la Base de datos

A continuación se explicarán algunas de las tablas principales utilizadas en este programa².

6.2.1. Tabla ticket

La tabla `ticket` es fundamental para el sistema de notificaciones automatizadas del bot de Discord. Esta tabla almacena la información principal de cada compra realizada en la tienda:

²Para ver toda la base de datos física así como su implementación completa, favor de visitar el repositorio del proyecto en GitHub: <https://github.com/Alvaro9669/PT.git>

Código 11: Estructura de la tabla ticket

```

1 CREATE TABLE ticket (
2   ID_ticket int NOT NULL AUTO_INCREMENT,
3   usuario_FK int DEFAULT NULL,
4   fecha date NOT NULL,
5   hora time NOT NULL,
6   total float NOT NULL,
7   notificar tinyint(1) DEFAULT '0',
8   PRIMARY KEY (ID_ticket),
9   KEY usuario_FK (usuario_FK),
10  CONSTRAINT ticket_ibfk_1 FOREIGN KEY (usuario_FK) REFERENCES usuarios
    (ID_usuarios) ON DELETE CASCADE,
11  CONSTRAINT ticket_chk_1 CHECK ((total >= 0))
12 ) ENGINE=InnoDB AUTO_INCREMENT=53 DEFAULT CHARSET=utf8mb4 COLLATE=
    utf8mb4_0900_ai_ci

```

El campo más importante para el funcionamiento del bot es **notificar**, un campo booleano que actúa como un sistema de persistencia para las notificaciones pendientes. Este campo se inicializa con valor 1 (true) cuando se crea un nuevo ticket tras una compra exitosa.

El bot de Discord consulta periódicamente esta tabla buscando tickets donde **notificar** = 1, indicando que aún no han sido notificados al usuario. Una vez que el bot envía exitosamente la notificación por mensaje directo, actualiza este campo a 0 (falso), marcando el ticket como ya notificado.

Esta implementación garantiza que, incluso si el bot se reinicia o experimenta fallos temporales, mantenga un registro persistente de qué tickets faltan por notificar. De esta manera, ninguna compra queda sin su respectiva notificación, funcionando como una cola de notificaciones pendientes con persistencia en la base de datos.

6.2.2. Tabla articulos_ticket

La tabla `articulos_ticket` almacena el detalle específico de los productos incluidos en cada ticket de compra:

Código 12: Estructura de la tabla articulos_ticket

```

1 CREATE TABLE articulos_ticket (
2   ID_articulist int NOT NULL AUTO_INCREMENT,
3   ticket_FK int DEFAULT NULL,
4   producto_FK int DEFAULT NULL,
5   cantidad int NOT NULL,
6   precio_compra float NOT NULL,
7   PRIMARY KEY (ID_articulist),
8   KEY ticket_FK (ticket_FK),
9   KEY producto_FK (producto_FK),
10  CONSTRAINT articulos_ticket_ibfk_1 FOREIGN KEY (ticket_FK) REFERENCES
    ticket (ID_ticket) ON DELETE CASCADE,
11  CONSTRAINT articulos_ticket_ibfk_2 FOREIGN KEY (producto_FK)
    REFERENCES producto (ID_producto) ON DELETE CASCADE,
12  CONSTRAINT articulos_ticket_chk_1 CHECK ((cantidad > 0)),
13  CONSTRAINT articulos_ticket_chk_2 CHECK ((precio_compra >= 0))
14 ) ENGINE=InnoDB AUTO_INCREMENT=99 DEFAULT CHARSET=utf8mb4 COLLATE=
    utf8mb4_0900_ai_ci

```

Una característica importante de esta tabla es el campo `precio_compra`, que almacena el precio exacto del producto al momento de la compra, no una referencia al precio actual en la tabla `producto`. Esta decisión de diseño es importante por las siguientes razones:

- **Integridad histórica:** Los tickets deben reflejar el precio real pagado por el cliente en el momento de la transacción.
- **Independencia de cambios futuros:** Si el precio de un producto se modifica posteriormente en el catálogo, esto no debe afectar el registro histórico de compras anteriores.
- **Transparencia con el cliente:** El usuario puede consultar exactamente lo que pagó, independientemente de fluctuaciones de precios posteriores.

Esta implementación asegura que cada ticket mantenga un registro inmutable y preciso de la transacción tal como ocurrió en su momento.

6.3. Filtro de productos

El sistema de filtrado de productos representa una implementación sofisticada de React que permite a los usuarios buscar y filtrar productos en tiempo real sin recargar la página. Esta funcionalidad aprovecha las características del Virtual DOM de React y el hook `useState` para proporcionar una experiencia de usuario fluida y eficiente.

6.3.1. Arquitectura del Sistema de Filtros

El sistema de filtros se basa en tres componentes principales de estado que trabajan de manera coordinada:

Código 13: Estados del sistema de filtros

```
1 // Estados para el sistema de filtros
2 const [filtros, setFiltros] = useState({
3   nombre: '', // Filtro por nombre de producto
4   categoria: '', // Filtro por categoria
5   subcategoria: '' // Filtro por subcategoria
6 });
7 const [categorias, setCategorias] = useState([]); // Lista de
   categorias disponibles
8 const [subcategorias, setSubcategorias] = useState([]); // Lista de
   subcategorias de la categoria seleccionada
9 const [productosFiltrados, setProductosFiltrados] = useState([]); //
   Productos despues de aplicar filtros
```

Este código define los estados principales del sistema de filtros. Cuando el usuario interactúa con cualquier filtro (busca por nombre, selecciona una categoría o subcategoría), estos estados se actualizan automáticamente. React detecta estos cambios y solo re-renderiza los componentes necesarios, no toda la página.

6.3.2. Filtro por Nombre de Producto

El filtro de búsqueda por nombre implementa una búsqueda en tiempo real que actualiza los resultados mientras el usuario escribe:

Código 14: Componente de búsqueda por nombre

```
1 // Componente de filtro de búsqueda
2 <div className="filtro-busqueda">
3   <input
4     type="text"
5     placeholder="Buscar productos..."
6     value={filtros.nombre}
7     onChange={(e) => handleFiltroChange('nombre', e.target.value)}
8     className="filtro-input"
9   />
10 </div>
```

Cuando el usuario escribe en el campo de búsqueda, se ejecuta automáticamente la función `onChange`, que actualiza el estado `filtros.nombre`. React detecta este cambio y activa el `useEffect` de filtrado, mostrando únicamente los productos que contienen el texto escrito.

La función `handleFiltroChange` gestiona las actualizaciones del estado de manera eficiente:

Código 15: Funcion de manejo de cambios en filtros

```
1 // Funcion para manejar cambios en los filtros
2 const handleFiltroChange = (campo, valor) => {
3   setFiltros(prev => {
4     const nuevosFiltros = { ...prev, [campo]: valor };
5
6     // Si cambia la categoria, limpiar subcategoria para evitar
7     // inconsistencias
8     if (campo === 'categoria') {
9       nuevosFiltros.subcategoria = '';
10    }
11    return nuevosFiltros;
12  });
13};
```

Esta función recibe qué filtro cambió y su nuevo valor, luego actualiza el estado correspondiente. Si el usuario cambia la categoría, automáticamente limpia la subcategoría para evitar conflictos.

6.3.3. Filtros Jerárquicos de Categorías

El sistema implementa un filtrado jerárquico donde las subcategorías dependen de la categoría seleccionada:

Código 16: Carga dinamica de subcategorias

```
1 // Efecto para cargar subcategorias cuando cambia la categoria
  seleccionada
2 useEffect(() => {
3   const fetchSubcategorias = async () => {
4     if (!filtros.categoria) {
5       setSubcategorias([]); // Limpiar subcategorias si no hay
        categoria seleccionada
6       return;
7     }
8     try {
9       const backendUrl = process.env.REACT_APP_BACKEND_URL || '
        http://192.168.1.116:5000';
10      const url = `${backendUrl}/api/categorias/${filtros.
        categoria}/subcategorias`;
11
12      const res = await fetch(url, {
13        method: 'GET',
14        headers: { 'Content-Type': 'application/json' },
15        credentials: 'include'
16      });
17
18      if (!res.ok) {
19        throw new Error('Subcategorias API error! status: ${res
        .status}');
20      }
21
22      const data = await res.json();
23      setSubcategorias(data);
24    } catch (error) {
25      console.error('Error fetching subcategorias:', error);
26      setSubcategorias([]);
27    }
28  };
29  fetchSubcategorias();
30 }, [filtros.categoria]); // Dependencia: solo se ejecuta cuando cambia
  la categoria
```

Cuando el usuario selecciona una categoría del menú desplegable, se actualiza `filtros.categori` y React detecta este cambio, ejecutando automáticamente este `useEffect`. El efecto hace una petición al servidor para obtener las subcategorías de esa categoría específica, actualiza la lista de subcategorías disponibles y llena el menú desplegable de subcategorías con las nuevas opciones.

6.3.4. Renderizado Optimizado con useEffect

Este es el `useEffect` más importante del sistema porque es el que realmente filtra los productos:

Código 17: Aplicacion de filtros en tiempo real

```
1 // Efecto para aplicar filtros a la lista de productos
2 useEffect(() => {
3     let productosFiltrados = productos;
4
5     // Filtrar por nombre de producto (busqueda de texto)
6     if (filtros.nombre.trim()) {
7         productosFiltrados = productosFiltrados.filter(producto =>
8             producto.n_articulo.toLowerCase().includes(filtros.nombre.
9                 toLowerCase())
10        );
11    }
12
13    // Filtrar por categoria seleccionada
14    if (filtros.categoria) {
15        productosFiltrados = productosFiltrados.filter(producto =>
16            producto.categoria_FK === parseInt(filtros.categoria)
17        );
18    }
19
20    // Filtrar por subcategoria seleccionada
21    if (filtros.subcategoria) {
22        productosFiltrados = productosFiltrados.filter(producto =>
23            producto.scategoria_FK === parseInt(filtros.subcategoria)
24        );
25    }
26
27    setProductosFiltrados(productosFiltrados);
28 }, [productos, filtros]); // Se ejecuta cuando cambian productos o
                             filtros
```

Cada vez que el usuario cambia cualquier filtro, React detecta que el estado `filtros` cambió y ejecuta automáticamente este `useEffect`. El efecto toma la lista completa de productos y aplica los filtros paso a paso: primero filtra por nombre si hay texto de búsqueda, luego por categoría si hay una seleccionada, y finalmente por subcategoría si también está seleccionada. Al final actualiza `productosFiltrados` con el resultado y React re-renderiza solo las tarjetas de productos que cambiaron de estado.

6.3.5. Función de Limpieza de Filtros

Para mejorar la experiencia del usuario, se incluye una función que permite restablecer todos los filtros:

Código 18: Funcion para limpiar filtros

```
1 // Funcion para limpiar todos los filtros aplicados
2 const limpiarFiltros = () => {
3   setFiltros({
4     nombre: '',
5     categoria: '',
6     subcategoria: ''
7   });
8 };
```

Cuando el usuario hace clic en el botón "Limpiar filtros", se ejecuta esta función que resetea todos los filtros a valores vacíos. React detecta que `filtros` cambió y ejecuta automáticamente el `useEffect` de filtrado. Como todos los filtros están vacíos, se muestran todos los productos originales. También se ejecuta el `useEffect` de subcategorías que, al no haber categoría seleccionada, limpia las subcategorías. El usuario ve todos los productos nuevamente y los menús desplegables vuelven a mostrar "Todas las categorías".

7. Resultados

En esta sección se presentan las interfaces del sistema, tanto del punto de venta como del bot de Discord. Estas interfaces son fundamentales para la interacción del usuario con el sistema, permitiendo realizar compras, gestionar productos y recibir notificaciones de manera eficiente y amigable.

7.1. Interfaces del Punto de Venta

Se presentarán las interfaces del punto de venta, que incluyen la cabecera, la página de productos y el carrito de compras.

7.1.1. Interfaz de Cabecera para Login

Esta interfaz mostrará la cabecera de la tienda virtual, esta cabecera es previa a que el usuario inicie sesión.



Figura 15: Cabecera de la aplicación con botón de inicio de sesión

La figura 15 muestra la cabecera de la aplicación, que incluye:

1. El botón de inicio, el cual llevará a la página principal de la tienda.
2. Un botón que desplegará una interfaz para acceder al servidor de Discord, la interfaz que abre se muestra en la Figura 17.

3. Un botón que permitirá al usuario comenzar el registro o login mediante Discord, esta interfaz redirigirá al usuario a figura 18.

7.1.2. Interfaz de Cabecera con Usuario Autenticado

Esta interfaz mostrará la cabecera de la tienda virtual, esta cabecera se mostrará una vez el usuario inicie sesión.



Figura 16: Cabecera de la aplicación con usuario autenticado

La figura 16 muestra la cabecera de la aplicación después del inicio de sesión de usuario. Los botones 1 y 2 de la cabecera no se modificarán, el botón 3 desaparecerá para ser reemplazado por:

- 3. El botón de carrito el cual le permitirá al usuario entrar a su interfaz de carrito.
- 4. Muestra el nombre del usuario y la imagen que tiene el usuario dentro de Discord.
- 5. Botón para cerrar sesión, el cual permitirá al usuario salir de su cuenta de Discord (se borra la cookie del usuario).

7.1.3. Interfaz de Invitación a Discord

La interfaz de invitación a Discord se mostrará al usuario una vez que dé clic en el botón 2 de la cabecera, este botón es el que permite al usuario unirse mediante código QR o el botón de invitación a Discord.



Figura 17: Interfaz de invitación a Discord.

En la figura 17 se pueden observar los siguientes elementos:

1. Un código QR que los usuarios pueden escanear para unirse al servidor de Discord.
2. Un botón que permite redirigir al usuario al servidor de Discord.

7.1.4. Interfaz de Autenticación

Esta interfaz muestra la ventana de autenticación de Discord que permite al usuario iniciar sesión en la aplicación.

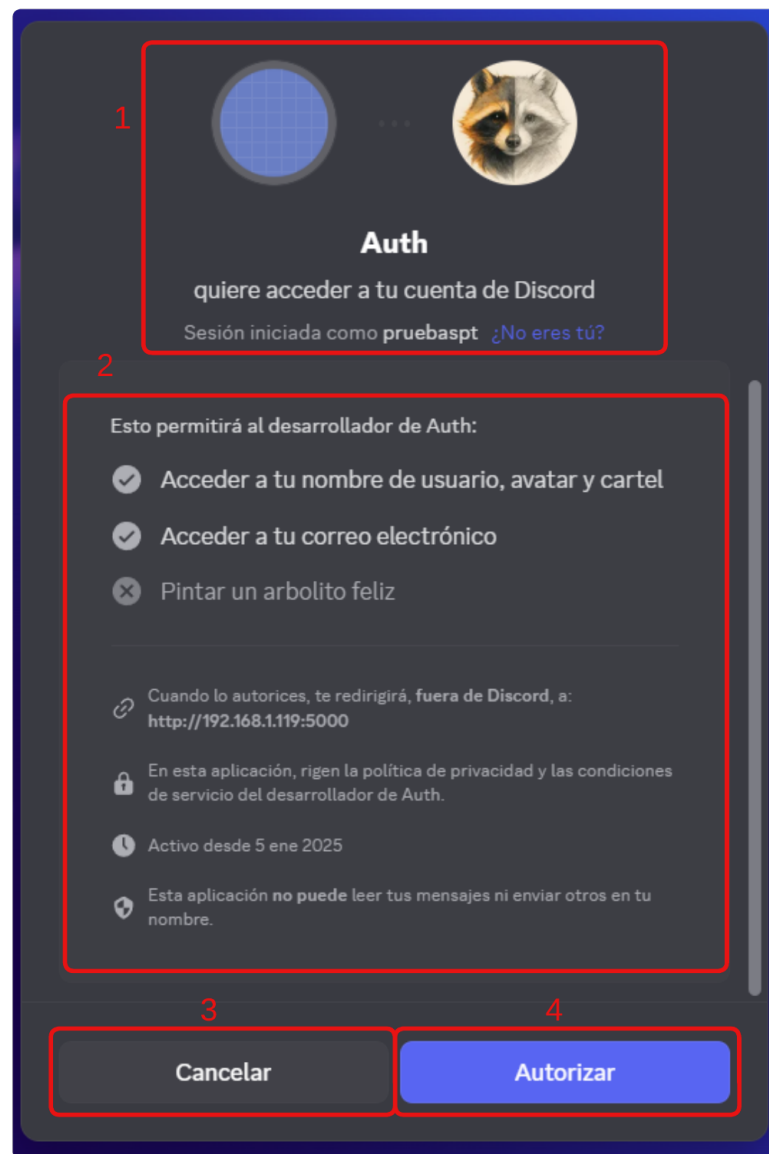


Figura 18: Vista de autorización de usuario

En la figura 18 se pueden observar los siguientes elementos:

1. Se muestra la imagen que tiene el la imagen y el nombre de usuario de discord que hara el inicio de sesión.
2. Se enlistan los permisos que va a requerir la aplicacion.
3. Boton que le permite al usuario cancelar el inicio de sesión.
4. Boton de autorizar que le permite al usuario iniciar sesión en la aplicación.

7.1.5. Interfaz del Carrito con Productos

Interfaz que se le mostrará al usuario cuando el carrito de compras tenga artículos seleccionados.

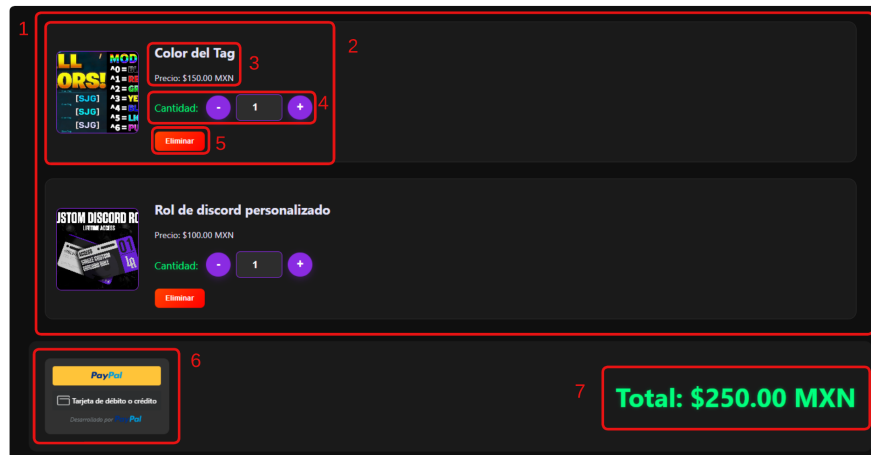


Figura 19: Vista del carrito de compras con productos

En la figura 19 se pueden observar los siguientes elementos:

1. Lista de productos agregados al carrito.
2. Tarjeta del producto
3. Nombre y precio del producto
4. Control de cantidad de los productos dentro del carrito
5. Botón de eliminar el producto del carrito
6. Botón para iniciar la compra vía PayPal.
7. Precio total de la compra a realizar.

7.1.6. Interfaz del Carrito Vacío

Interfaz que se le mostrará al usuario cuando el carrito de compras esté vacío.

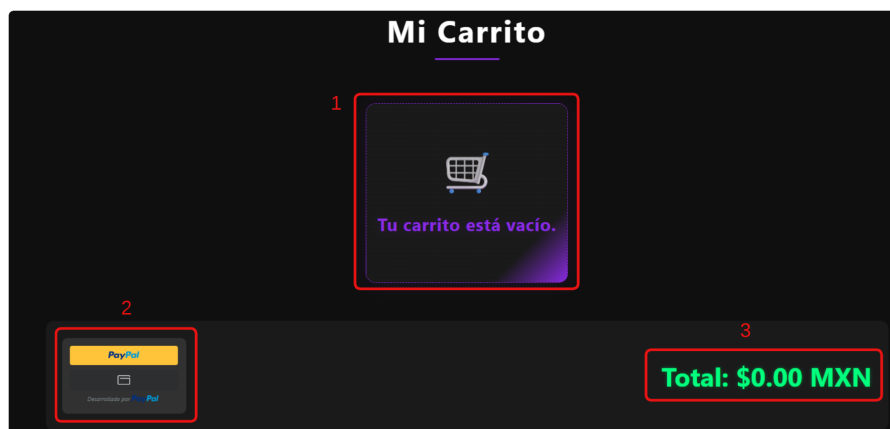


Figura 20: Vista del carrito de compras vacío

En la figura 20 se pueden observar los siguientes elementos:

1. Botón con mensaje indicando que el carrito está vacío.
2. Botones de compra con PayPal, no dejará realizar compra si no hay artículos.
3. Total del valor de los artículos en el carrito

7.1.7. Interfaz de búsqueda de Productos

En esta interfaz se mostrara la interfaz para que el usuario pueda comenzar a buscar productos y agregarlos al carrito

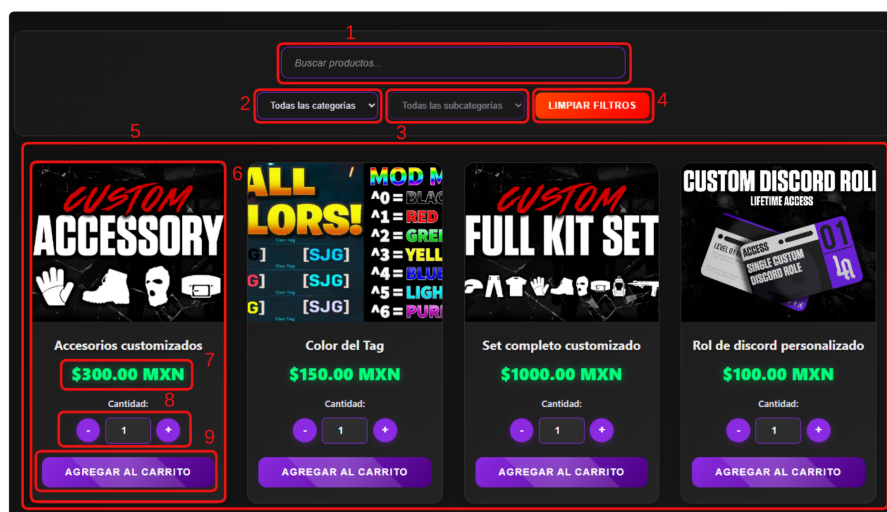


Figura 21: Vista de la lista de productos y su filtro de búsqueda

En la figura 21 se pueden observar los siguientes elementos:

1. Barra de búsqueda de productos por comparación de nombre ya sea parcial o total.
2. Menú desplegable de las categorías.
3. Menú desplegable de las subcategorías.
4. Botón para reiniciar los filtros de búsqueda.
5. Contenedor de los productos.
6. Tarjeta de cada producto con su información básica (imagen, nombre, precio, etc.).
7. Precio del producto.
8. Control de cantidad para el producto.
9. Botón para agregar el producto al carrito.

7.2. Interfaces del Bot de Discord

En esta sección se mostraran las interfaces del bot así como las interfaces de notificaciones via discord.

7.2.1. Creación de Comandos para Usuarios

Instrucción que escribe el usuario en el chat de Discord.

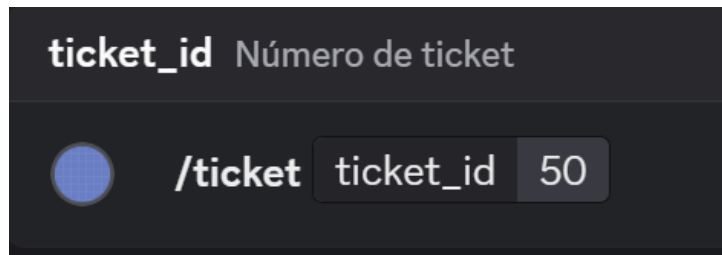


Figura 22: Comandos registrados del bot

La figura 22 muestra el comando ticket que contiene el comando /ticket seguido de el ticket_id y el número de ticket que el usuario quiera visualizar.

7.2.2. Comprobación de Tickets Pendientes

Es la interfaz del bot donde se podrá observar la comprobación constante que hace sobre los tickets pendientes así como la creación de canales para los tickets específicos.

```
1[NOTIFICACIONES] Tickets pendientes: 0
2[NOTIFICACIONES] Tickets pendientes: 1
3[DEBUG] Intentando crear canal para ticket ID: 53
4[DEBUG] Creando canal de ticket para ID: 53
5[DEBUG] Canal creado exitosamente: ticket-53
6[DEBUG] Buscando ticket ID: 53 (Tipo: number)
7Notificación enviada por DM a pruebaspt
8[NOTIFICACIONES] Tickets pendientes: 0
```

Figura 23: Verificación de tickets pendientes

En la figura 23, se pueden ver las siguientes acciones del bot:

Línea 1. Comprueba que no haya ningún ticket pendiente si el contador es 0 entonces no hay tickets por notificar.

Línea 2. Si el contador de tickets es 1 entonces comenzará el proceso de notificación automática.

Línea 3. Realiza las peticiones a la BD para comprobar los datos del ticket que no ha sido notificado.

Línea 4. Comienza la creación del canal para el ticket.

Línea 5. Crea exitosamente el canal privado con el usuario dueño del ticket.

Línea 6. Realiza la petición nuevamente para rescatar la información completa del ticket.

Línea 7. Envía un mensaje directo al usuario de que su canal para el ticket ha sido creado.

Línea 8. Una vez realizada la notificación el ticket cambia su estado a notificado y el bot continúa realizando la comprobación de tickets pendientes.

7.2.3. Interfaz de Notificación vía Discord

Interfaz de Discord donde el usuario podrá revisar su canal privado y la información de su ticket.



Figura 24: Notificación de ticket al usuario

- En la figura 24, se muestra la interfaz de discord con las siguientes características:
1. Canales generales del servidor de Discord como los de mensajería y los de voz.
 2. Categoría de tickets donde se crearán los canales para el ticket del usuario.
 3. Información del ticket del usuario como nombre del comprador, fecha, hora, artículos comprados, precio, cantidad y total.
 4. Boton que le permite al usuario o administrador cerrar el ticket.

7.2.4. Envío de Notificaciones por Canales Privados

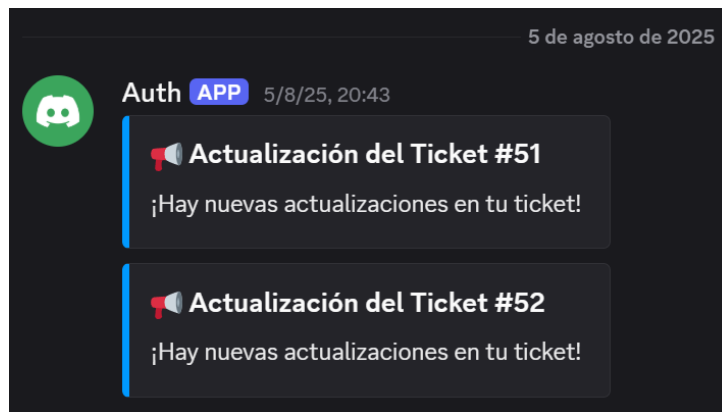


Figura 25: Mensaje privado de notificación

La figura 25 ilustra cómo el bot envía notificaciones directas a los usuarios a través de mensajes privados en Discord. Estas notificaciones pueden incluir información sobre el estado de un pedido, recordatorios de pago o promociones especiales.

8. Conclusiones

Durante el desarrollo de este proyecto se enfrentaron varios retos, especialmente el aprendizaje y uso de tecnologías nuevas como React para el frontend y la librería Discord.js para la integración del bot. Fue necesario revisar detalladamente la documentación para Discord.js que se encuentra en el Discord Developer Portal [10] y la documentación oficial de React [11]. Estos recursos fueron fundamentales para comprender cómo interactuar con la API de Discord y cómo construir una interfaz de usuario dinámica y responsiva.

Uno de los aspectos más importantes para este proyecto fue el diseño de la base de datos ya que tras varios rediseños en la base de datos se lograron integrar todas las funcionalidades requeridas para su correcto funcionamiento. También se requirió conseguir una serie de llaves API para poder interactuar con los servicios externos como Discord y PayPal.

Si bien se lograron incorporar todas las funcionalidades planteadas para este proyecto, sí faltaron algunas características que podrían mejorar la experiencia del usuario, como la creación de una interfaz para que el usuario pueda revisar todas las compras que ha realizado o un comando para Discord que le permita al usuario agregar productos al carrito desde Discord. Estas mejoras se consideran para futuras iteraciones del proyecto.

Un aspecto importante a considerar es la limitación impuesta por Discord en cuanto al envío de mensajes y notificaciones por parte de los bots. Discord establece límites estrictos en la cantidad de mensajes que un bot puede enviar en un período de tiempo determinado (rate limits). Por ejemplo, según la documentación oficial de Discord [12], los bots pueden enviar hasta 5 mensajes por segundo por canal, y existen límites globales y por usuario que, si se superan, pueden llevar a que el bot sea temporalmente bloqueado o incluso baneado. Esto implica que, en escenarios donde se requiera notificar a un gran número de usuarios simultáneamente, el sistema debe implementar mecanismos de control y espera para evitar superar estos límites y garantizar la continuidad del servicio.

9. Referencias

Referencias

- [1] PayPal. Bienvenido a PayPal. Disponible en: <https://www.paypal.com/mx/webapps/mpp/welcome-to-paypal>. Accedido: 21-jun-2025.
- [2] MEE6. MEE6 - El bot de Discord más completo. Disponible en: <https://mee6.xyz/es>. Accedido: 21-jun-2025.
- [3] R. Teja Rubio. Proyecto Tienda Virtual: Uniformes Escolares. UOC, 2016. Disponible en: <https://openaccess.uoc.edu/handle/10609/49345>
- [4] Facebook. Marketplace de Facebook. Disponible en: <https://www.facebook.com/marketplace/>. Accedido: 21-jun-2025.
- [5] AliExpress. Disponible en: <https://es.aliexpress.com/>. Accedido: 21-jun-2025.
- [6] Campaign Monitor. .^Email Marketing Benchmarks and Statistics for 2023, Campaign Monitor, 2023. Disponible en: <https://www.campaignmonitor.com/resources/guides/email-marketing-benchmarks/>. Accedido: 21-nov-2024.
- [7] Return Path. "2023 Deliverability Benchmark Report," Validity, 2023. Disponible en: <https://www.validity.com/resource-center/state-of-email-2023/>. Accedido: 21-nov-2024.
- [8] Mobile Marketing Association. "Mobile Marketing Statistics and Trends 2023," MMA Global, 2023. Disponible en: <https://www.mmaglobal.com/research/mobile-marketing-statistics-2023>. Accedido: 21-nov-2024.
- [9] WhatsApp Business. "WhatsApp Business API: Transforming Customer Communication," Meta for Business, 2023. Disponible en: <https://business.whatsapp.com/products/business-api>. Accedido: 21-nov-2024.
- [10] Discord Developer Portal. "Discord.js Documentation," Discord, 2024. Disponible en: <https://discord.js.org/#/docs/discord.js/stable/general/welcome>. Accedido: 15-dic-2024.
- [11] React Team. React Documentation," Meta, 2024. Disponible en: <https://react.dev/learn>. Accedido: 15-dic-2024.
- [12] Discord Developer Portal. Rate Limits," Discord, 2024. Disponible en: <https://discord.com/developers/docs/topics/rate-limits>. Accedido: 15-dic-2024.